Experiment 4 (BCS551) – DQL and Sorting Data

4.1 DQL (Data Query Language)
DQL is used to retrieve data from database tables. The SELECT statement is the primary command. It extracts data without modifying the database, making it safe and non-destructive. SELECT may be combined with clauses like WHERE, ORDER BY, GROUP BY, etc., to refine output.

4.2 WHERE Clause
The WHERE clause filters rows in a SELECT statement based on conditions. It supports operators like =, <>, >, <, BETWEEN, IN, LIKE, and IS NULL. This enables conditional retrieval and selective extraction of data.

4.3 Pattern Matching (LIKE Operator)
LIKE is used to search patterns in text data. Wildcards used:
% : any number of characters
_ : exactly one character
Example: last_name LIKE 'A%' retrieves names starting with A.

4.4 ORDER BY Clause
ORDER BY sorts records in ascending (ASC) or descending (DESC) order. It allows sorting based on multiple columns. It is always the last clause in a SELECT query.

4.5 Substitution Variables (&)
Substitution variables allow runtime input in Oracle SQL using &var and &&var.
&var prompts every time, whereas &&var prompts only once and reuses the value.

Experiment 5 (BCS551) – Subquery and Nested Query

5.1 What is a Subquery?
A subquery is a query inside another SQL query. The outer query uses the output of the inner query to perform additional operations. Subqueries must be enclosed in parentheses.

5.2 Types of Subqueries
a) Single-Row Subquery:
Returns exactly one row. Uses operators like =, <, >, <=.
b) Multiple-Row Subquery:
Returns multiple rows. Uses IN, ANY, ALL, EXISTS.
c) Correlated Subquery:
Depends on a value from the outer query to execute. Executes row by row.

5.3 Nested Subqueries
A nested subquery is one that contains another subquery inside it. It is useful for multi-level filtering.

5.4 Correlated Subqueries
These subqueries reference columns from the outer query, making them dependent on the outer result for each evaluation.

5.5 Subquery Operators
IN: Checks if a value exists in a list returned by the subquery.
ANY: Compares with any value from the subquery's result.
ALL: Compares with all values from subquery output.
EXISTS: True if subquery returns at least one row.

EXPERIMENT – 6

Aim

To implement different types of joins (CROSS JOIN, NATURAL JOIN, INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN) on the employees and departments tables.

Theory

In a relational database, related data is stored across multiple tables. **Joins** are used to combine rows from two or more tables based on a related column (usually a foreign key–primary key relationship).

- **CROSS JOIN**
  Produces the Cartesian product of two tables. Every row from the first table is combined with every row from the second table.
  Result size = rows(table1) × rows(table2).
- **NATURAL JOIN**
  Automatically joins tables based on **columns with the same name** and compatible data types. No join condition is written explicitly.
- **INNER JOIN**
  Returns only the rows where the join condition matches in both tables.
- **LEFT OUTER JOIN**
  Returns all rows from the left table and matching rows from the right table. Non-matching rows from the right side are shown as NULL.
- **RIGHT OUTER JOIN**
  Returns all rows from the right table and matching rows from the left table. Non-matching rows from the left side are shown as NULL.
- **FULL OUTER JOIN**
  Returns all rows when there is a match in either left or right table. Non-matching rows from both sides are shown as NULL.

Joins are essential for working with normalized databases where data is spread over multiple related tables.

EXPERIMENT – 7

Aim:

To analyze and summarize order/payment data using aggregate functions (SUM, AVG, MIN, MAX, COUNT) along with the GROUP BY and DISTINCT clauses.

Theory:

Structured databases store multiple records, and in many cases, we don't need individual rows — we need summarized information.
GROUP BY groups rows that contain the same value in one or more columns.
Aggregate functions operate on these groups to return a single result per group.

Common aggregate functions:

Function	Purpose
SUM()	Total value of a numeric column
AVG()	Average value
MIN()	Lowest value
MAX()	Highest value
COUNT()	Returns number of rows

DISTINCT helps find the number of unique values in a field.
GROUP BY is used when we want aggregate results **category-wise**, while calculations without GROUP BY give results for the **entire table**.

EXPERIMENT – 8

Aim:

To study and implement various single row functions in SQL using the DUAL table and the EMPLOYEES database.

Theory:

Single row functions are built-in SQL functions that operate on **one row at a time** and return **one result per row**. They are widely used to format, convert, extract and calculate data within SQL queries. These functions do not modify the actual stored data in the table; they only affect the output of the query.

Single row functions are categorized into the following types:

1. **Character Functions:** Used to manipulate text values.
   Examples: UPPER(), LOWER(), INITCAP(), LENGTH(), SUBSTR(), INSTR(), REPLACE()
2. **Numeric Functions:** Used to perform arithmetic formatting on numeric data.
   Examples: ROUND(), TRUNC(), MOD()
3. **Date Functions:** Used to manipulate and calculate values based on dates.
   Examples: SYSDATE, ADD_MONTHS(), LAST_DAY(), MONTHS_BETWEEN()
4. **Conversion Functions:** Used to convert data from one datatype to another.
   Examples: TO_CHAR(), TO_DATE(), TO_NUMBER()
5. **General Functions:** Useful for handling special cases such as NULL values.
   Examples: NVL(), NVL2(), COALESCE()

Single row functions are very important for data formatting and reporting because they allow output customization without changing the stored data.

EXPERIMENT – 9

Aim:

To study and implement **Views** and **Indexes** in SQL by creating views (with and without CHECK OPTION), selecting data using views, and dropping views, along with understanding the purpose of indexing.

Theory:

A **View** is a virtual table created from the result of a SQL query. It does not store data physically; instead, it displays data stored in underlying base tables. Views help simplify complex queries, improve security by restricting access to specific columns or rows, and enhance data abstraction.

Types of views used in this experiment:

1. **Simple View (Without CHECK OPTION):**
    - Displays selected data from a table.
    - Allows insertion of rows that may not satisfy the view condition.
2. **View With CHECK OPTION:**
    - Ensures that rows inserted or updated through the view must satisfy the view condition.
    - Prevents invalid modifications.

Advantages of Views:

- Restricts access to sensitive data
- Simplifies query usage
- Reduces query complexity
- Provides logical data independence

A **Database Index** is a data structure used to improve the speed of data retrieval. An index is created on one or more columns of a table to make searching faster, similar to an index in a book.

Advantages of Indexes:

- Faster searching and sorting
- Improved performance of SELECT queries
- Useful for columns frequently used in WHERE, JOIN, and ORDER BY clauses

Note: Indexes increase query speed but slightly slow down INSERT and UPDATE operations because the index must also be updated.

EXPERIMENT – 10

Aim:

To implement and execute PL/SQL database programming constructs including **Triggers**, **Procedures**, **Functions**, and **Cursors**, and understand their role in data validation, automation and record processing.

Theory:

PL/SQL (Procedural Language for SQL) extends SQL by providing procedural capabilities. It allows writing programs that run inside the database server and enhance automation, security and control over data operations.

1. Trigger

A **Trigger** is a stored PL/SQL block that executes automatically in response to DML operations such as INSERT, UPDATE or DELETE. Triggers are commonly used for:

- Data validation
- Activity logging
- Enforcing business rules

Example concept in this lab:
A trigger was written on the **DEPT** table to ensure that the DEPTNO column **cannot have NULL or duplicate values**. If violation occurs, an error is raised automatically.

2. Procedure

A **Procedure** is a named PL/SQL block that performs a task and may receive input parameters. It does not return a value using RETURN, but output parameters may be used if needed. Procedures are useful for automating frequently used operations like updating records.

3. Function

A **Function** is similar to a procedure but **must return a single value** using the RETURN statement. It can be used inside SQL statements if it is free from

database state modification. Functions are useful for calculations like computing annual salary.

4. Cursor

A **Cursor** is a pointer to the result set of a query used for row-by-row data processing.
A **cursor FOR LOOP** handles cursor declaration, opening, fetching and closing automatically. Therefore, programmers do not need to manually fetch each row.